Chennai Mathematical Institute

M.Sc. Computer Science

# Cost-Aware Verification in Velvet

A dissertation submitted in partial fulfillment
of the requirements for the degree of

Master of Science in Computer Science

by

**Huidrom Ronald Mangang**

May 2026

# Acknowledgements

My time at CMI as a master's student in computer science has been relatively short but nonetheless has been some of my most memorable periods of my life so far and for that I owe my thanks to many people.

First of all, I wish to express my deepest gratitude to my thesis advisors, Dr. S. P. Suresh and Dr. Madhavan Mukund, for their guidance and support throughout my thesis work.

I thank some of my best friends I met at CMI who I spent some of my most memorable times with—Lucky, Kathakali, Maulik, Gyanshree and Romit. I also thank a junior friend Titan. Life was never dull with them and they gave me a homely feeling at a place quite far away from my home.

I also thank my parents for their constant love, support and belief in me.

*For my grandmother, Pashot, who passed away while I was working on my thesis.*

**Abstract**

Velvet is a Dafny-style verifier for imperative programs in Lean, built on Loom. This thesis extends Velvet with cost-aware verification, enabling programmers to specify and verify resource bounds alongside functional correctness. The extension is implemented via syntax-directed instrumentation using Lean's elaboration mechanism, allowing cost reasoning to reuse the existing verification infrastructure. We demonstrate the approach on some examples.

# Contents


**1 Introduction** **2**

**2 A foundational multi-modal verifier** **6**
- 2.1 Framework . . . 6
- 2.2 Shallow embedding . . . 8
- 2.3 Hybrid workflow . . . 9
- 2.4 Velvet . . . 10
- 2.5 Imperative programming syntax . . . 11
- 2.6 Multi-modal verification . . . 13
- 2.7 More on Loom and Velvet . . . 16

**3 Extending Velvet with cost-aware verification** **17**
- 3.1 Motivation . . . 17
- 3.2 Cost as a Mutable Variable . . . 19
- 3.3 Elaboration-Based Cost Instrumentation . . . 21
- 3.4 A Demonstration . . . 23

**4 A Fibonacci case study** **26**

**5 Practical ramifications** **31**
- 5.1 On recursive functions . . . 31
- 5.2 On loop invariants . . . 33

**6 Limitations and future work** **36**
- 6.1 Limitations . . . 36
- 6.2 Future Work . . . 38

**7 Conclusion** **40**

# Chapter 1

# Introduction

Consider the following function definition:

```
def divide (x : ℕ) (y : ℕ) (h : y ≠ 0) := x / y
```

The declaration above is the usual integer division operation on natural numbers, except that the function takes an additional *proof* argument. By `x : ℕ` we mean that `x` is a term of type `ℕ`; the third parameter `h` has type `y ≠ 0`, a proposition that depends on the value of `y`. In other words, the function is not just *typed* but *indexed* by a proposition: it is a dependent function.

Under the Curry–Howard correspondence (propositions-as-types, proofs-as-terms), a term of type `y ≠ 0` is exactly a witness (a proof) that `y` is nonzero. Thus `divide` can only be applied when the caller supplies such a witness. For example:

```
#eval divide 10 2 (by decide)
```

Here `by decide` constructs the required proof term (a certificate that `2 ≠ 0`), and the evaluation proceeds without any runtime check.

Now consider this inductive type:

```
inductive NonEmptyList (α : Type) where
| mk : (xs : List α) → xs ≠ [] → NonEmptyList α
```

The `inductive` declaration defines `NonEmptyList α` as the type of non-empty lists over `α`. Its single constructor, `NonEmptyList.mk`, has the dependent type

```
NonEmptyList.mk : (xs : List α) → xs ≠ [] → NonEmptyList α
```

which reads: given a list `xs : List α` and a proof of `xs ≠ []`, construct an element of `NonEmptyList α`. The second argument is a *proof obligation* that enforces the invariant "the list is not empty" at the type level.

We can now implement a safe head projection:

```
def NonEmptyList.head {α} : NonEmptyList α → α
| .mk (x :: _) _ ⇒ x
| .mk [] h ⇒ (h rfl).elim
```

The pattern match is exhaustive because every `NonEmptyList α` is built with `.mk`. In the first clause the payload has the form `x :: _` and we return the head `x`. The second clause is logically impossible: it matches the empty list `[]` while `h` is a proof of `[] ≠ []`. Applying `h` to `rfl` (the canonical proof of judgmental equality `[] = []`) yields a term of type `False`; invoking `(h rfl).elim` eliminates the contradiction (by `False.elim`) and thus produces a term of the expected return type. In short, the impossible branch is discharged by logical contradiction rather than by a runtime error.

These examples illustrate a central design point of dependent-type proof assistants: invariants that would otherwise require runtime checks are encoded in the type system and discharged by supplying proof terms. By making propositions first-class and allowing types to depend on terms, Lean (based on the Calculus of Inductive Constructions) moves many correctness obligations from runtime into the type checker. The payoff is strong: programs and proofs can be composed so that certain classes of runtime failures are provably impossible.

While Lean is an extremely powerful theorem prover and programming language, its functional style and theorem-proving oriented workflow can initially feel unfamiliar to many programmers. A large majority of programmers today are used to imperative languages such as C, C++, Java or Python, where programs are written as sequences of state-changing commands. While Lean certainly supports programming, directly writing imperative verification-oriented programs inside pure Lean syntax can be somewhat cumbersome for those unfamiliar with dependent type theory or functional programming.

At the same time, Lean provides something very attractive: a small trusted kernel, expressive theorem proving, metaprogramming support and the ability to reason about programs inside the same environment in which the programs are written. Thus there is a strong motivation to somehow bridge these two worlds: on one side, familiar imperative-style programming and verification, and on the other side, the foundational guarantees and proof capabilities of Lean.

Loom, developed by the Verified Systems Engineering Lab at NUS School of Computing, is an attempt in this direction. Rather than being a single verifier, Loom is a framework for building foundational multi-modal verifiers inside Lean. The goal is to provide a general infrastructure where programs, semantics, execution, testing and theorem proving can coexist within the same environment. In

particular, Loom aims to combine automated verification techniques such as SMT solving together with interactive theorem proving using Lean tactics.

We shall later see Velvet, which is one such verifier built on top of Loom. Velvet provides an imperative-style DSL inspired by Dafny while still retaining the benefits of Lean underneath. This allows programmers to write programs in a more familiar syntax while simultaneously obtaining access to foundational verification and interactive proofs.

In Chapter 2, we introduce Loom [GPZ+26] and discuss its design, semantics and overall framework. We then introduce Velvet and discuss its syntax, verification workflow and major features. In Chapter 3, we discuss the main contribution of this thesis which extends Velvet with cost-aware verification. We explain the design decisions behind the extension and demonstrate the implementation through small examples.

In Chapter 4, we study a larger Fibonacci case study where we verify recursive cost bounds using the developed extension. This chapter also demonstrates the interaction between automated solving and interactive theorem proving in Lean.

In Chapter 5, we discuss some practical ramifications and consequences of the design choices made in this thesis, particularly regarding recursive functions and loop invariants. In Chapter 6, we discuss limitations of the current system and several possible future directions. Finally, Chapter 7 concludes the thesis with a brief summary and recap of the preceding chapters.

Henceforth, we shall not cover preliminary material on Lean in much detail. Indeed, the thesis assumes some basic familiarity with Lean syntax, tactics and its programming model. The focus here is not Lean itself but rather Loom, Velvet and the cost-aware verification extension built on top of them.

For readers unfamiliar with Lean, [dMU21] gives a good overview of Lean 4 as both a theorem prover and a functional programming language. It discusses the design philosophy of Lean and how programming and proving coexist in the same environment. Since Loom and Velvet are themselves deeply embedded into Lean, understanding Lean's foundations and design gives a clearer picture of why such verifier frameworks can be implemented in the first place.

For a more practical introduction, [Chr24] provides an accessible and engaging entry point into functional programming in Lean. In particular, it is useful for understanding monads, dependent types and programming patterns which appear repeatedly throughout Loom and Velvet. Since Velvet programs are ultimately elaborated into Lean terms and monadic programs, familiarity with these concepts becomes quite helpful.

On the theorem proving side, [AdMKU25] is an excellent reference for tactics, in-

teractive proving and proof development in Lean. Much of Velvet's hybrid workflow relies on smoothly transitioning between automated reasoning and manual interactive proofs. Therefore, understanding Lean tactics and proof states is important for understanding how verification proceeds once automation alone is insufficient.

Finally, the Lean language reference [Lea25b] and the Lean 4 metaprogramming book [PTA+25] are valuable resources regarding Lean's elaboration mechanism, syntax extensions and metaprogramming infrastructure. These become especially relevant later in the thesis when we discuss how the cost-aware verification extension was implemented by modifying the elaboration layer rather than changing the underlying semantics or verification framework. In particular, Lean's metaprogramming facilities play a central role in making such extensions feasible with relatively small and localized changes.

The code for the work reported in this thesis can be found in the repository: https://github.com/rhuid/velvet.

# Chapter 2

# A foundational multi-modal verifier

## 2.1 Framework

Loom is a foundational verifier in that all its semantics and proofs are expressed in a theorem prover and proof assistant (Lean). Therefore, in this sense, the final guarantees rely only on the correctness of the Lean kernel. This shifts the burden of trust from an external verification pipeline to a small trusted core. Unlike traditional verification systems where one must trust a large standalone verifier implementation, Loom reduces this trusted base to the theorem prover itself. Since Lean has a very small and carefully checked kernel, this provides a stronger level of confidence in the correctness of the verification results.

To be precise, Loom is actually a framework for building multi-modal verifiers. That is to say, it is not itself a single verifier for one fixed language, but rather a meta-framework from which many verifiers can be constructed. By abuse of language (and for convenience), we shall often refer to Loom itself as a verifier, although technically it is a framework for constructing such verifiers. Later, we shall see Velvet, which is an actual verifier built on top of Loom.

One of the central goals of Loom is to combine several styles of verification within a single foundational setting. In particular, Loom supports multiple verification modes such as concrete execution and testing, symbolic execution, SMT-assisted automation, and interactive theorem proving. This combination is important because different verification tasks are often better suited to different methodologies. Lightweight automated proving may be sufficient for simple properties, while more difficult proofs may require interactive reasoning inside Lean itself. Loom provides a common semantic framework in which these approaches coexist naturally.

Loom provides mathematical constructions together with formal proofs of these constructions expressed directly in Lean. Programs in Loom are modeled using monads. Informally, monads provide a way to structure computations as a sequence of steps, where each step may not only produce a value but may also involve computational effects such as mutable state, nondeterminism, failure, or IO-like behavior. Rather than treating programs merely as syntactic objects, Loom gives them a mathematical semantics directly inside Lean. Programs therefore become objects that can themselves be reasoned about formally.

An important advantage of this monadic approach is compositionality. The meaning of a larger program is determined systematically from the meanings of its components. This gives Loom a modular semantic structure: complex behaviors can be constructed from simpler pieces while preserving formal reasoning principles. Such compositional semantics are particularly useful in verification, since proofs about small program fragments can be combined to reason about larger programs.

Loom further uses an algebraic construction known as a monad transformer. A monad transformer may be viewed as a type-level construction which receives one monad and produces another monad enriched with additional computational effects. In practice, one may think of each monad transformer as encoding some particular program behavior such as mutable state, failure, nondeterminism, exceptions, or tracing.

By stacking monad transformers together, one can build semantics for programs with multiple interacting effects. For example, one transformer may introduce nondeterministic execution while another introduces mutable state. The resulting combined monad then models programs containing both effects simultaneously. In Loom, verifiers themselves are constructed in this compositional way.

This transformer-based design is one of the reasons Loom is highly extensible. Rather than implementing a verifier monolithically, one can build new verifiers by assembling and composing semantic components. This allows Loom to support a wide variety of verification styles while still remaining foundational.

Another important feature of Loom is that it uses shallow embedding inside Lean. Instead of defining an entirely separate programming language together with an interpreter inside Lean, Loom programs are represented directly using Lean constructs and semantics. Consequently, Loom programs are executable Lean programs themselves. This shallow embedding greatly simplifies integration with Lean's existing infrastructure, automation, and proof mechanisms.

Because Loom is embedded directly inside Lean, it also inherits Lean's powerful metaprogramming and elaboration facilities. This makes Loom highly malleable and extensible. New verification features and language extensions can often be added by transforming or elaborating syntax into existing semantic constructions. This flexibility plays an important role in the cost-aware verification extension developed

in this thesis.

## 2.2 Shallow embedding

Lean is a very malleable language. Indeed, it is quite feasible to extend Lean or build a DSL inside Lean. One of Lean's major strengths is that it provides not only a theorem proving environment but also facilities for language design and metaprogramming. This makes it possible to construct verification-oriented languages while still remaining entirely inside Lean itself.

Generally speaking, there are two approaches to building a language on top of Lean. These are deep embedding and shallow embedding.

In deep embedding, we represent the syntax of the language explicitly using data structures (or inductive types in our case). Programs are represented as abstract syntax trees (ASTs). One then writes an interpreter, compiler, or semantic function which gives meaning to these syntax trees. In other words, the language being verified exists as data inside Lean.

While this approach is flexible, it also introduces additional proof burden. Since the semantics are implemented manually through an interpreter, one must also reason about and verify the correctness of this interpreter itself. Consequently, a substantial amount of effort goes into proving that the implemented semantics actually correspond to the intended semantics of the language.

Loom instead uses shallow embedding. Here, programs are written directly using Lean's own language and semantic machinery. There is no need to define a completely separate syntax together with an external interpreter. Programs are simply Lean terms themselves.

This gives several important advantages. First, Loom automatically reuses Lean's type system, elaboration machinery, parser, and proof infrastructure. Secondly, programs now have direct access to Lean tactics, theorem proving features, and automation tools. One can seamlessly move between automated verification and interactive proving without leaving the Lean environment.

Another important advantage is executability. Since Loom programs are directly represented as Lean programs, they can also be executed inside Lean itself. This allows testing, symbolic execution, and deductive reasoning to coexist within the same framework.

The shallow embedding approach also makes Loom highly extensible. Since programs are fundamentally Lean syntax, one can transform or elaborate them using Lean's metaprogramming facilities. New language constructs and verification features can therefore be introduced with relatively small modifications. This becomes particularly relevant in the cost-aware verification extension developed in

this thesis, where Lean's elaboration mechanism is used to automatically insert cost instrumentation into Velvet programs.

Finally, shallow embedding leads to a smoother integration between programming and proving. Instead of translating between an external verifier and a theorem prover, the programs, specifications, and proofs all coexist in a single unified environment. This considerably simplifies experimentation and extension of the verification framework.

## 2.3 Hybrid workflow

Another strong aspect of Loom is its hybrid verification workflow. Loom aims to seamlessly combine automated reasoning through SMT solving with interactive theorem proving using Lean tactics and `by`-blocks. Rather than forcing the user to choose between full automation and fully manual proofs, Loom allows both approaches to coexist naturally within the same environment.

In many practical verification tasks, a large portion of proof obligations are routine and can be discharged automatically by SMT solvers such as Z3 or CVC5. For these cases, Loom provides automated tactics which attempt to solve proof goals with minimal user intervention. This makes the verification process closer to the experience of tools such as Dafny, where simple correctness conditions can often be verified automatically.

However, full automation is not always sufficient. Complex arithmetic properties, inductive arguments, recursive reasoning, or domain-specific lemmas may require more detailed proofs which automated solvers fail to discover on their own. In traditional SMT-based verifiers, such situations often become a dead end for the user. Loom instead allows one to directly fall back to Lean's interactive theorem proving capabilities.

Because Loom is embedded inside Lean, the user can continue the proof manually using ordinary Lean tactics, lemmas, and proof scripts. Automated and interactive reasoning therefore become part of the same workflow rather than two completely separate systems.

This hybrid design is particularly attractive because it combines the strengths of both worlds. SMT solvers are efficient at routine logical reasoning and constraint solving, while interactive theorem proving provides expressiveness and flexibility for difficult proofs. Loom allows users to benefit from automation whenever possible while still retaining the full expressive power of Lean when automation alone is insufficient.

Another advantage of this unified workflow is that execution, testing, verification, and proving all happen inside the same environment. Programs can be

executed concretely, symbolically reasoned about, automatically verified, and interactively proved without translating between multiple external tools. This leads to a smoother and more integrated verification experience.

In practice, this also lowers the barrier to experimentation. One may begin with testing or lightweight automated verification and later strengthen the proof interactively only where necessary. This incremental style of verification is one of the appealing aspects of Loom as a foundational verification framework.

## 2.4 Velvet

We have mentioned before that Loom is not a single verifier but rather a framework for building verifiers. We have been loosely referring to Loom as if it were itself a verifier language, but now we arrive at Velvet, an actual DSL and verifier built on top of Loom and Lean.

The syntax of Velvet is inspired by Dafny and other imperative verification languages. Velvet allows one to write imperative-style programs together with their specifications directly inside Lean. Programs can be annotated with preconditions, postconditions, loop invariants, termination measures and so on, much like in traditional program verifiers.

One important point is that Velvet is shallowly embedded in Lean. Programs are not translated into some external verification language. Rather, Velvet programs are ultimately Lean terms themselves, benefiting directly from Lean's type system, elaboration machinery, and theorem proving environment.

Velvet is also the name of the monad in which all Velvet programs are written. Indeed, Velvet defines the abbreviation

```
abbrev VelvetM α := NonDetT DivM α
```

That is, `VelvetM` is constructed from the divergence monad `DivM` together with the non-determinism monad transformer `NonDetT`.

The divergence monad models computations which may terminate or diverge (that is, computations which may fail to terminate). This is important for reasoning about general recursive or potentially non-terminating programs.

The non-determinism transformer models computations which may have multiple possible execution paths. Such semantics are particularly useful for symbolic execution, specification-oriented reasoning, and modeling abstract program behaviors.

Consequently, every Velvet program ultimately has type `VelvetM`. Programs therefore carry semantic meaning directly inside Lean's logic.

Velvet supports a hybrid verification workflow. Verification usually begins with automation through the `loom_solve` tactic provided by Loom. This tactic attempts to automatically discharge proof obligations generated from program specifications.

For many simple or routine verification tasks, this automation is already sufficient. More powerful automation is additionally available through external SMT solver backends such as Z3 and CVC5.

However, automation is not always enough. Some proofs may require inductive arguments, auxiliary lemmas, arithmetic manipulations, or domain-specific reasoning which SMT solvers cannot discover automatically. Since Velvet is embedded inside Lean, the user can seamlessly continue the proof interactively using ordinary Lean tactics.

This gives a verification workflow combining both automation and interactive reasoning. One typically attacks proof obligations using automation first and then finishes remaining goals interactively if necessary.

Another attractive feature is that Velvet programs are executable. Since the programs are Lean terms, they can be evaluated directly inside Lean itself. Thus, execution, testing, symbolic reasoning, and deductive verification all coexist within the same foundational environment.

## 2.5 Imperative programming syntax

A majority of programmers are used to imperative-style programming languages such as C, C++, or Java, and much of programming in current industry still follows the imperative paradigm. Lean, however, is fundamentally a functional language. One appealing aspect of Velvet is that it provides an imperative-style verification language on top of Lean while still retaining Lean's foundational semantics and theorem proving capabilities.

This imperative syntax is made possible through Lean's powerful elaboration and metaprogramming facilities. From the user's perspective, programs look close to ordinary imperative code, but underneath, they are elaborated into Lean terms in the Velvet monad.

The following is a typical program written in Velvet.

```
method DoubleAll (mut arr: Array Int) return (u: Unit)
    ensures arr.size = arrOld.size
    ensures ∀ i, 0 ≤ i ∧ i < arr.size → arr[i]! = 2 * arrOld[i]!
    do
    let mut i := 0
    while i < arr.size
        invariant arr.size = arrOld.size
        invariant 0 ≤ i ∧ i ≤ arr.size
        invariant ∀ k, 0 ≤ k ∧ k < i → arr[k]! = 2 * arrOld[k]!
        invariant ∀ k, i ≤ k ∧ k < arr.size → arr[k]! = arrOld[k]!
        done_with i = arr.size
    do
        arr := arr.set! i (2 * arr[i]!)
        i := i + 1
    return
```

The program doubles every element of the array while formally specifying and verifying its correctness.

The function begins with the keyword `method`. Preconditions and postconditions are specified using `require` and `ensures` clauses. Multiple specifications can be given simply by writing multiple clauses.

Loops are annotated using `invariant`. These invariants describe properties which must remain true throughout loop execution and are essential for verification. Velvet also supports optional clauses such as `done_with` and `decreasing` for reasoning about loop termination and final states.

Mutable parameters are declared using the keyword `mut`. When a parameter is mutable, Velvet automatically makes its original value available using the `Old` suffix. In the above example, `arrOld` refers to the original array before mutation. This is particularly useful when specifying properties relating the final program state to the initial state.

Although the syntax looks imperative, the program is ultimately elaborated into an ordinary Lean term in the monad `VelvetM`. Indeed, if we inspect the generated definition using

```
#print DoubleAll
```

we obtain

```
def DoubleAll : Array ℤ → VelvetM (Unit × Array ℤ)
```

The type above may initially appear slightly unusual, particularly the extra `Array ℤ` appearing in the return type. This is due to how Velvet models mutable

state. Since mutable parameters may be updated during execution, their final values are returned alongside the ordinary return value of the method. This allows specifications to reason about both the initial and final program states in a purely functional setting.

One important point here is that the imperative syntax is only surface-level syntax. Underneath, the program still has a purely functional semantics inside Lean. Mutation and loops are ultimately encoded using ordinary Lean constructs and monadic operations. This allows Velvet to provide an imperative programming experience while still remaining within Lean's foundational logical framework.

## 2.6 Multi-modal verification

To execute or run a Velvet program, we can simply evaluate it inside Lean using `#eval`. For example,

```
#eval (DoubleAll #[1, 2, 3]).run
```

This executes the program concretely on the given input array. Since Velvet programs are ultimately Lean terms, they are executable directly inside Lean itself. This is already one attractive aspect of the framework: the same program can both execute and later be formally verified.

Beyond ordinary execution, Velvet also supports property-based testing (PBT). Property-based testing allows us to test programs automatically against many randomly generated inputs before attempting full deductive verification.

An example setup for property-based testing is shown below.

```
import CaseStudies.TestingUtil

-- Extract executable version
extract_program_for insertionSort
prove_precondition_decidable_for insertionSort
prove_postcondition_decidable_for insertionSort
derive_tester_for insertionSort

-- Run property-based tests
run_elab do
  let g : Plausible.Gen (_ × Bool) := do
    let arr ← Plausible.SampleableExt.interpSample (Array Int)
    let res := insertionSortTester arr
    pure (arr, res)
  for _ in [1: 500] do
    let res ← Plausible.Gen.run g 10
    unless res.2 do
      IO.println s!"postcondition violated for input {res.1}"
      break
```

Here, random arrays are generated automatically and the generated test cases are checked against the method specifications. In this sense, specifications are not only used for formal verification but also serve as executable test oracles.

Of course, testing alone is not a proof. Even if a program passes thousands of random tests, there may still exist some unseen input violating the specification. Property-based testing therefore serves mainly as a lightweight and practical debugging tool before full verification.

To prove that the specifications hold for all possible inputs, we instead use deductive verification. In Velvet, this is done using

```
prove_correct DoubleAll by
  loom_solve
```

The `prove_correct` command generates verification conditions from the program together with its specifications. The tactic `loom_solve` then attempts to automatically discharge these proof obligations.

As mentioned earlier, `loom_solve` usually forms the first line of attack. For many routine verification problems, especially involving arithmetic, arrays, invariants, and symbolic reasoning, the automation is often sufficient by itself.

When automation is insufficient, the user can seamlessly continue the proof interactively using ordinary Lean tactics. This is one of the major advantages of

Velvet being embedded directly inside Lean. One is not restricted to only SMT automation or only interactive theorem proving. Instead, both approaches coexist smoothly in the same environment.

For example, automation may reduce a difficult proof obligation into a few smaller mathematical goals which can then be finished manually using induction, rewriting, arithmetic tactics, or custom lemmas.

Another verification mode supported by Loom and Velvet is symbolic execution. Rather than executing the program on concrete inputs, symbolic execution reasons about symbolic values and explores execution paths symbolically. One can think of this as running the program on unknown values while tracking logical constraints generated by branching conditions.

For instance, in a conditional statement such as

```
if x > 0 then
  y := x + 1
else
  y := 0
```

symbolic execution explores both branches:

- Path 1: assume `x > 0`, then conclude `y = x + 1`
- Path 2: assume `x ≤ 0`, then conclude `y = 0`

The resulting symbolic conditions are then used in verification.

The term *multi-modal verification* refers precisely to this coexistence of multiple ways of reasoning about programs within a unified framework.

In Loom and Velvet, one can:

- execute programs concretely,
- perform random testing through property-based testing,
- reason symbolically through symbolic execution,
- invoke SMT-based automation,
- and finish proofs interactively using Lean tactics.

Importantly, all of these happen within the same foundational environment. The same program, the same specifications, and the same semantics are reused across all modes of reasoning. This unification is one of the central ideas and major strengths of Loom.

## 2.7 More on Loom and Velvet

Velvet can do a lot more than what we have described in the above. For instance, it allows programmers to change the semantics (that is from "partial" to "total" correctness for termination, or from "demonic" to "angelic" for non-deterministic programs). These options are set in the beginning of the program file. Usually, the options are set to "partial" correctness and "demonic" choice.

Switching from "partial" to "total" correctness restricts programmers to write only terminating programs in the sense that when they write loops, for instance, they also have to provide termination measures.

For more details, one can refer to their excellent documentation [Ver26].

# Chapter 3

# Extending Velvet with cost-aware verification

## 3.1 Motivation

So far, the focus of verification has mainly been on *functional correctness*. That is, we verify that programs produce the correct outputs, maintain invariants, preserve specifications, and do not violate intended behavior.

However, functional correctness alone is often not sufficient.

In many practical situations, we are not only interested in whether a program is correct, but also whether it is efficient enough or consumes acceptable amounts of resources. A program may be perfectly correct functionally while still being too expensive to run in practice.

For example, an algorithm may terminate and produce correct results, yet have an unacceptable running cost due to poor complexity. Similarly, in resource constrained environments such as embedded systems, blockchain smart contracts, or real-time systems, resource usage itself becomes an important correctness concern.

Indeed, some systems explicitly associate execution with cost. For instance, Ethereum smart contracts use a notion of *gas*, where every operation has an associated execution cost. Programs exceeding the available gas budget fail to execute. In such settings, reasoning about cost is not merely an optimization problem but part of program correctness itself.

This motivates extending verification beyond ordinary functional properties and towards reasoning about resource usage.

In this work, we focus on adding cost-aware verification to Velvet. More precisely, we extend Velvet so that programmers can specify and verify cost bounds directly alongside ordinary functional specifications.

Our goal is not merely to track cost internally, but to expose cost as a verifiable logical quantity which can appear in specifications and proofs.

The following were the main desiderata guiding the design of the extension.

- **Minimal changes to the existing Velvet infrastructure.**
  We wanted the extension to integrate naturally into the already existing Velvet verification pipeline rather than redesigning the system entirely.
  One possible approach would have been to modify the underlying semantics of Velvet or redesign the monadic infrastructure itself. However, this would make the extension significantly more invasive and would require reworking larger parts of the framework.
  Instead, the aim was to preserve as much of the existing system as possible. Programs already written in Velvet should continue to work naturally with only small additions to specifications when cost reasoning is desired.
  In particular, we wanted the extension to reuse the already existing machinery for mutable variables, specifications, verification condition generation and proof automation.
- **Automation of cost tracking.**
  Another major goal was that programmers should not manually manage cost updates throughout their code.
  Ideally, programmers should write programs exactly as before and only specify cost bounds through ordinary specification clauses such as `ensures` and loop `invariant`s.
  The actual bookkeeping of cost increments should happen automatically underneath the language.
  For example, the programmer should not need to manually write statements such as

  ```
  cost := cost + 1
  ```

  after every operation.
  Doing so would be tedious, error-prone, and would clutter the program with implementation details unrelated to the actual algorithm being expressed.
  Instead, we wanted cost accounting to be inserted automatically during elaboration while preserving the original user-facing syntax as much as possible.
  Thus, from the user's perspective, cost verification should feel like an extension of ordinary Velvet verification rather than an entirely different programming style.
- **Preserving the multi-modal nature of Velvet.**
  Another important design goal was that the extension should continue to support not only deductive verification but also execution and testing.

Since Velvet programs are executable Lean terms, extending the system with cost reasoning should not break ordinary execution on concrete inputs.
Similarly, property-based testing should continue to work naturally.
In particular, we wanted cost-aware programs to still support:

- ordinary execution using `#eval`,
- property-based testing on random inputs,
- symbolic reasoning and verification,
- SMT-assisted automation,
- and interactive theorem proving.

Thus, the extension should preserve the overall philosophy of Loom and Velvet as a foundational multi-modal verification framework.

The resulting design attempts to satisfy these goals while requiring only modest changes to the surface syntax and verification infrastructure of Velvet.

## 3.2 Cost as a Mutable Variable

The central idea behind the extension is surprisingly simple: we model cost as a mutable variable inside Velvet itself.

Rather than introducing a completely separate resource verification framework or changing the semantics of the verifier in a major way, we instead reuse the already existing machinery for mutable state already present in Velvet.

This turns out to fit quite naturally with how Velvet handles mutable variables.

Recall that in Velvet, mutable parameters are automatically returned alongside the ordinary return value of the method. For example, a method such as

```
method Foo (mut arr : Array Int) return (u : Unit)
```

is elaborated into something which also returns the updated value of `arr`. This is precisely why expressions such as `arrOld` become available in postconditions and loop invariants. The verifier internally keeps track of both the original and final states of mutable variables.

The extension exploits exactly this mechanism.

We introduce a mutable variable called `cost`. Since mutable variables are already threaded through the computation and returned automatically, the updated value of `cost` also becomes available at the end of execution.

This immediately gives us a convenient way to write cost specifications.

For example, we can now write specifications such as

```
ensures cost - costOld ≥ arr.size
```

or

```
ensures cost - costOld ≤ n * n
```

directly inside the ordinary Velvet specification language.

In other words, cost becomes just another logical quantity that the verifier can reason about.

This approach has several advantages.

First, it integrates very smoothly with the already existing verification machinery of Velvet. Since mutable variables were already supported, we can reuse the same ideas for specifications, verification condition generation and proof obligations.

Second, the approach is conceptually simple. Rather than introducing an entirely new semantics for resource usage, we merely extend the existing notion of state with one additional mutable quantity.

Third, cost specifications now look very natural from the user's perspective. They are written exactly like ordinary postconditions and loop invariants.

For example, inside a loop one may write invariants such as

```
invariant cost - costOld ≥ i
```

or similar bounds relating cost to the progress of the algorithm.

Another useful aspect of this design is that the cost becomes visible during ordinary execution as well.

Since the updated cost is returned as part of the final result, we may also run programs concretely and inspect their exact execution cost for particular input values.

For instance, evaluating a program on a concrete input now not only produces the ordinary result but also the accumulated execution cost.

This gives the framework a lightweight profiling-like capability directly inside the verifier.

Of course, this also means that method calls internally receive and return the additional `cost` parameter.

Conceptually, one can think of the execution state carrying along a running cost counter which is updated throughout execution.

At first this may appear somewhat inconvenient, since every function call now has an extra value being threaded through it. However, most of this bookkeeping is hidden from the user by the elaboration layer, so programs can still be written in a mostly ordinary style.

Moreover, this additional parameter is actually useful rather than merely being an implementation artifact. Because the cost is explicit, one can inspect, execute and reason about it directly.

Another important question is where exactly the cost should be incremented.

To perform cost verification, we must decide which program constructs contribute to execution cost and by how much.

For example, one may choose to increment cost for assignments, conditionals, loop iterations, function calls, array accesses, or other atomic operations.

Different cost models are possible depending on what one wants to measure.

In our case, the important point is that Velvet programs contain only finitely many syntactic constructs. Therefore, one can systematically specify how cost is updated for each relevant construct.

This keeps the implementation manageable. We do not need to reason about an unbounded collection of language features; rather, we only need to extend the handling of the existing Velvet syntax appropriately.

The actual insertion of these cost updates is handled automatically underneath the language during elaboration, which we discuss in the next section.

## 3.3 Elaboration-Based Cost Instrumentation

One of the main reasons Lean is well suited for this project is that it is not only a theorem prover but also a very programmable language environment. This means that one can extend the language itself instead of only proving things about a fixed language. In particular, Lean provides metaprogramming and elaboration facilities, and these are exactly what make our cost-aware extension possible.

Elaboration is the stage where what the user writes is turned into the actual Lean term that will later be checked by the kernel. In other words, the user writes a relatively high-level piece of syntax, and Lean elaborates it into a fully explicit term with all the missing details filled in. This happens after parsing and before kernel type checking. So the kernel does not see the surface syntax directly; it sees only the elaborated term. This is important because it means that if we can control elaboration, then we can control what the verifier actually sees, while keeping the surface language pleasant for the user.

This is where Lean becomes especially useful for our purpose. Instead of asking the user to manually write cost updates everywhere, we can intercept the Velvet program during elaboration and automatically transform it into a cost-aware version. The user still writes the program in the usual Velvet style, but underneath the elaborator inserts the extra cost bookkeeping steps.

The changes we made are centered around exactly this idea.

First, we ensure that the program has a mutable cost binder. If the user has already provided one, we reuse it. Otherwise, we inject a mutable variable named `cost`. This is done so that cost becomes part of the mutable state handled by Velvet,

just like the other mutable parameters.

Second, we identify the mutable variables and keep track of their "old" versions. This is important because mutable variables in Velvet are returned at the end of execution, and their original values can then be referenced in postconditions and invariants using the `Old` suffix. In our case, cost is treated in the same way, so the updated cost can be used directly in the specification.

Third, we traverse the Velvet `do`-notation structure and insert cost increments at the appropriate places. This is done by expanding the different constructs appearing in the program: assignments, function calls, conditionals, matches, and loops. In each case, we decide whether the construct should count as a cost step and then automatically add the corresponding update.

For example, an assignment statement is rewritten so that the cost is incremented before the actual assignment. Similarly, a conditional or a match expression can be charged when the branch is evaluated, and a loop body can be charged once per iteration. The exact choice of where to charge cost depends on the intended cost model, but the important point is that this is handled systematically and uniformly by the elaboration process.

One subtle point is that some constructs need special care because they do not explicitly return a value in the surface syntax. For example, a `pure` statement producing `PUnit.unit` should not accidentally turn into a meaningful returned tuple of mutable state. That is why the elaborator has to distinguish between ordinary terms and control-flow statements and handle them accordingly. This kind of detail is precisely the sort of thing Lean's metaprogramming support makes manageable.

Another important aspect of the implementation is that loops are handled by recursively elaborating the loop body and then inserting the cost charge in the right place. Since Velvet loops are themselves elaborated into Lean terms, we can extend them without changing the core verification engine. This is a significant advantage of using Lean: the syntax, the semantics, and the proof infrastructure are all close enough that we can modify one part without having to redesign everything else.

At the end of this process, the program that the user wrote has effectively been turned into a cost-instrumented Velvet program, while the user still interacts with the original, cleaner syntax. The verification pipeline then sees a program where cost is already part of the state, and therefore ordinary postconditions and loop invariants can reason about it directly.

So the overall picture is this: Lean gives us a programmable elaboration layer, and we use that layer to automatically enrich Velvet programs with cost tracking while preserving the original user-facing syntax as much as possible. This is what makes the extension lightweight, reusable, and practical.

## 3.4 A Demonstration

Consider the following program.

```
-- division by multiple subtractions
method div_mod (mut a : Nat) (b : Nat) return (q : Nat)
  require b > 0
  ensures aOld = q * b + a
  ensures a < b
  ensures cost ≥ costOld + 3 * q
  do
    let mut q := 0
    while b ≤ a
    invariant a + q * b = aOld
    invariant cost ≥ costOld + 3 * q
    do
      a := a - b
      q := q + 1
    return q
```

This program computes the quotient and remainder of a division using repeated subtraction. The variable `a` is declared mutable and is repeatedly reduced by subtracting `b`. At the same time, the quotient variable `q` is incremented. When the loop terminates, the remaining value of `a` is the remainder and `q` is the quotient.

The functional correctness specifications are expressed in the usual way using `ensures`. The postcondition

```
ensures aOld = q * b + a
```

states the standard division property. Since `a` is mutable, its original value can be referred to as `aOld`. The final values of `q` and `a` therefore satisfy the equation

$$a_{\text{old}} = q \cdot b + a$$

which is precisely the expected relation between quotient and remainder.

The second postcondition

```
ensures a < b
```

states that the final remainder is strictly smaller than the divisor, which is again part of the standard specification of integer division.

More interesting for us is the cost specification:

```
ensures cost ≥ costOld + 3 * q
```

This expresses a lower bound on the execution cost of the program. Observe that the programmer never manually increments the cost variable anywhere in the code. The only place where cost appears is in specifications. The actual insertion of cost increments is handled automatically during elaboration.

Intuitively, each iteration of the loop performs several operations such as:

- checking the loop condition,
- subtraction `a := a - b`,
- incrementing `q := q + 1`,
- assignments and control flow.

The elaborator injects cost updates into the generated Lean code according to the chosen accounting model. In this example, the specification claims that at least three units of cost are consumed per iteration of the loop. Since the loop runs exactly `q` times, the final lower bound becomes

$$\texttt{cost} \geq \texttt{costOld} + 3q.$$

The same style naturally extends to loop invariants. The invariant

```
invariant cost ≥ costOld + 3 * q
```

tracks the accumulated cost during execution. This is important because the proof of the final postcondition is usually carried through the loop invariant. In practice, cost invariants behave very similarly to ordinary functional invariants.

One advantage of this approach is that execution still works normally. Since `cost` is simply another mutable parameter underneath, we can supply an initial concrete value and evaluate the program directly.

```
#eval (div_mod 20 8 0).run
```

This executes the program concretely with initial cost `0`. The output now contains not only the computed quotient and remainder, but also the final cost. Hence, the same mechanism supports both abstract verification and concrete cost observation.

Finally, to prove both functional correctness and the cost specification, we use the ordinary Velvet verification pipeline.

```
prove_correct div_mod by
  loom_solve
```

For a simple example such as this, the automation tactic `loom_solve` is already sufficient to discharge all proof obligations automatically. No special proof machinery is needed for cost verification beyond the additional specifications themselves.

This example demonstrates an important aspect of the extension: cost reasoning is integrated into the existing workflow rather than treated as a separate system. Programs are still executed, tested and verified in the same way as before, except that specifications can now additionally talk about resource usage.

# Chapter 4

# A Fibonacci case study

Here we look at a more interesting example involving recursion. Besides showing cost-aware verification on recursive functions, this example also illustrates how automation and interactive theorem proving work together inside the same environment.

We begin with the following setup.

```
import CaseStudies.Velvet.Std
import CaseStudies.TestingUtil

set_option loom.semantics.termination "partial"
set_option loom.semantics.choice "demonic"
set_option loom.solver "grind"
```

These settings are compulsory for our development.

The option

```
set_option loom.semantics.termination "partial"
```

tells Loom to use partial correctness semantics rather than total correctness semantics. Under partial correctness, if the program terminates, then the specification must hold. However, termination itself is not required to be proved. This is convenient for examples such as recursive Fibonacci where we wish to focus mainly on cost reasoning rather than proving termination.

If instead we used

```
set_option loom.semantics.termination "total"
```

then Loom would additionally require a termination measure together with a proof that recursive calls decrease according to that measure.

Next, we set

```
set_option loom.semantics.choice "demonic"
```

Loom models programs using non-deterministic semantics. Under demonic choice, all possible executions of the program must satisfy the specification. This is the standard interpretation used in program verification because we wish to guarantee correctness under every possible behavior.

Finally,

```
set_option loom.solver "grind"
```

sets the default backend used by `loom_solve`. Here we use Lean's builtin `grind` tactic. Of course, Loom can also use SMT solvers such as Z3 or CVC5, but for this example the builtin automation is already sufficient for a large part of the proof.

Next, we define Fibonacci in the ordinary functional style.

```
def fibonacci (a : Nat) : Nat :=
  match a with
  | 0 ⇒ 0
  | 1 ⇒ 1
  | a + 2 ⇒ fibonacci a + fibonacci (a + 1)
```

This definition is purely mathematical. It is not the program we verify. Instead, it acts as a specification-level function which we later use in cost bounds and proofs.

Next, we define the actual executable and verifiable Velvet program.

```
method fibo (a : Nat) return (res : Nat)
  ensures cost - costOld ≥ fibonacci (a + 1)
  do
  if a ≤ 1 then
    return a
  else
    let ⟨prev₁, cost'⟩ ← fibo (a - 1) cost
    let ⟨prev₂, cost''⟩ ← fibo (a - 2) cost'
    cost := cost''
    return prev₁ + prev₂
```

The specification we want is

```
cost - costOld ≥ fibonacci (a + 1)
```

That is, the total cost of running `fibo a` should be at least the `(a + 1)`-th Fibonacci number.

Observe also the explicit threading of the `cost` variable through the recursive calls.

```
let ⟨prev₁, cost'⟩ ← fibo (a - 1) cost
let ⟨prev₂, cost''⟩ ← fibo (a - 2) cost'
cost := cost''
```

This is an issue discussed later (next chapter) regarding recursive functions. The updated cost returned from one recursive call must be passed into the next call and then propagated back into the current scope.

To finish the verification, we require a small auxiliary lemma about Fibonacci.

```
lemma fib_step (a : Nat) (h : a ≥ 1) :
    fibonacci (a + 1) = fibonacci a + fibonacci (a - 1) := by
  cases a with
  | zero ⇒ grind
  | succ a ⇒ cases a <;> simp [fibonacci]; grind
```

This lemma is mathematically obvious from the recursive definition of Fibonacci. However, the verifier still requires it explicitly because the automation does not automatically unfold recursive definitions in all situations.

Next, we add the following two lines.

```
attribute [grind] fib_step
set_option maxHeartbeats 1000000
```

The command

```
attribute [grind] fib_step
```

registers the lemma with the `grind` automation tactic. This means that whenever `grind` encounters goals involving Fibonacci expressions, it may automatically attempt to use `fib_step` during proof search.

The second line

```
set_option maxHeartbeats 1000000
```

increases Lean's resource limit for tactic execution. Proof automation in Lean works under a bounded computational budget called "heartbeats". Sometimes recursive arithmetic reasoning can exhaust the default limit, especially when using tactics like `grind` and `omega`. Increasing the heartbeat limit simply gives the automation more time to search for a proof.

Finally, we can attempt the verification.

```
prove_correct fibo by
  loom_solve
  { simp
    cases a with
    | zero => simp [fibonacci]
    | succ b => cases b <;> try (simp [fibonacci]) <;> contradiction }
  { rcases i_2 with rfl
    rcases i_5 with rfl
    have hsum : costOld + 1 + fibonacci a + fibonacci (a - 1) ≤ i_4 := by sorry

    have hfib : fibonacci a + fibonacci (a - 1) = fibonacci (a + 1) := by
      simpa using (fib_step a (by omega)).symm

    have hfinal : costOld + 1 + fibonacci (a + 1) ≤ i_4 := by grind
    omega
    }
```

This proof nicely demonstrates the hybrid workflow of Velvet.

First, we invoke

```
loom_solve
```

which automatically discharges a large part of the verification conditions. However, the automation alone is unable to completely finish the proof. At this point we seamlessly switch into ordinary Lean tactics and continue interactively.

The following lines

```
rcases i_2 with rfl
rcases i_5 with rfl
```

perform substitutions using equalities generated internally by the verifier. Essentially, these simplify tuples and state components introduced during the elaboration process.

We then prove an intermediate arithmetic bound:

```
have hsum : costOld + 1 + fibonacci a + fibonacci (a - 1) ≤ i_4 := by sorry
```

At this point, we temporarily used `sorry`. The remaining proof obligation is actually straightforward.

Indeed, if we inspect the goal state at this point, Lean shows:

```
Goals (1)
case «cost - costOld ≥ fibonacci (a + 1)»
a costOld : ℕ
if_neg : ¬a ≤ 1
i i_1 i_3 i_4 : ℕ
ensures_1 : (i, i_1).2 - (costOld + 1) ≥ fibonacci (a - 1 + 1)
ensures_1_1 : (i_3, i_4).2 - i_1 ≥ fibonacci (a - 2 + 1)
⊢ i_4 - costOld ≥ fibonacci (a + 1)
```

The two hypotheses

```
ensures_1
ensures_1_1
```

are precisely the induction hypotheses obtained from the two recursive calls. Intuitively, they state that the recursive calls already satisfy the required cost bounds.

By combining these two inequalities and simplifying the arithmetic expressions, we obtain the desired intermediate bound

```
costOld + 1 + fibonacci a + fibonacci (a - 1) ≤ i_4
```

The next step is to rewrite the Fibonacci expression using our auxiliary lemma.

```
have hfib : fibonacci a + fibonacci (a - 1) = fibonacci (a + 1) := by
  simpa using (fib_step a (by omega)).symm
```

This transforms the accumulated cost expression into exactly the form required by the specification.

Finally, the proof concludes with

```
have hfinal : costOld + 1 + fibonacci (a + 1) ≤ i_4 := by grind
omega
```

The `grind` tactic performs the rewriting and simplification, while `omega` finishes the remaining Presburger arithmetic reasoning.

This example demonstrates several important aspects simultaneously:

- recursive cost-aware verification,
- interaction between automation and manual proofs,
- use of auxiliary lemmas,
- arithmetic reasoning using `omega`,
- and how verification conditions generated by Loom can be inspected and completed interactively.

More importantly, it shows the general philosophy behind Velvet and Loom: automation first, followed by interactive theorem proving only when necessary.

# Chapter 5

# Practical ramifications

## 5.1 On recursive functions

One somewhat undesirable consequence of the modifications we made appears when we write recursive functions. Recall that our approach treats `cost` as a ghost variable which is automatically threaded through every function written using the `method` keyword. Internally, this is implemented by making `cost` a mutable parameter. While this gives a very convenient mechanism for writing and verifying cost specifications, it also introduces some syntactic overhead when recursive calls are involved.

To see this, consider the following factorial function written under "total" correctness semantics.

```
set_option loom.semantics.termination "total" in
method FactorialTotal (n: Nat) return (res: Nat)
  ensures res > 0
  do
    if n = 0 then
      return 1
    else
      let ⟨prev, cost'⟩ ← FactorialTotal (n - 1) cost
      cost := cost'
      return n * prev
  termination_by n
  decreasing_by omega -- Prove termination measure decreases
```

Since we are working with total correctness semantics, we must also provide a termination measure together with a proof that the recursive calls decrease with respect to that measure. This is standard in theorem proving environments and is independent of our cost extension.

The important part for us is the recursive call itself:

```
let ⟨prev, cost'⟩ ← FactorialTotal (n - 1) cost
cost := cost'
```

Observe what is happening here. The current value of `cost` must be passed explicitly as an argument to the recursive call. The recursive call then returns both the actual return value and the updated cost value. We then manually update the current `cost` variable using

```
cost := cost'
```

This pattern has to be repeated for every recursive call. Conceptually, this is not difficult, but syntactically it becomes somewhat unpleasant and verbose. Indeed, it somewhat breaks the illusion that the programmer need not worry about cost bookkeeping.

Ideally, one would like to write recursive calls in the much simpler style

```
let prev ← FactorialTotal (n - 1)
```

or at least something very close to this. In other words, we would like the cost threading to happen implicitly behind the scenes rather than appearing directly in user programs.

It is important to note, however, that this issue is largely syntactic rather than foundational. The underlying verification infrastructure remains perfectly sound and functional. The problem is mainly one of ergonomics and user experience. Since Lean is highly extensible through elaboration and metaprogramming, there is no fundamental obstacle preventing us from hiding this plumbing from the user.

Indeed, one possible improvement would be to modify the elaboration process so that recursive calls automatically thread the cost variable internally. The user would then write recursive programs in essentially the same style as before, while the elaborator silently inserts the necessary cost passing and updates. This would preserve the simplicity of the original Velvet syntax while still supporting cost verification.

Unfortunately, due to time constraints, I have not implemented this additional layer of syntax sugar. Nevertheless, the issue does not affect the core design or correctness of the extension itself. The essential cost-verification mechanism already works, and the remaining inconvenience is mostly a matter of surface-level syntax refinement.

## 5.2 On loop invariants

Now with the implementation of cost-aware verification, we are required to almost always specify additional loop invariants describing how the `cost` variable evolves across iterations. Of course, this is only needed when we actually wish to verify cost bounds. If no postcondition involving `cost` is written, then no such invariant is necessary.

However, once we begin specifying cost bounds in postconditions, the verifier must somehow connect the final cost value with the behavior of the loop. In particular, it must understand how much cost is accumulated after every iteration. Since the verifier reasons inductively over loops using invariants, this information must also be supplied inductively in the form of loop invariants.

For instance, consider the following simple program which increments every entry of an array by a fixed value.

```
method incrementArray (mut arr : Array Nat) (inc : Nat) return (res : Unit)
require costOld = 0
ensures ∀ j < arr.size, arr[j]! = arrOld[j]! + inc
ensures cost ≥ arr.size
do
  let mut i := 0
  while i < arr.size
  invariant i ≤ arr.size
  invariant ∀ j < i, arr[j]! = arrOld[j]! + inc
  invariant ∀ j ≥ i, j < arr.size → arr[j]! = arrOld[j]!
  invariant cost ≥ costOld + i
  do
    arr[i] := arr[i]! + inc
    i := i + 1
  return ()
```

The functional part of the specification states that every array entry has been incremented by `inc`. The final postcondition involving cost states that the total cost is at least the size of the array. Intuitively, this makes sense because we perform at least one meaningful update operation per iteration.

The important invariant here is

```
invariant cost ≥ costOld + i
```

which captures the idea that after processing `i` elements, the accumulated cost has increased by at least `i`. This acts as the inductive bridge between the loop body and the final postcondition.

Indeed, if we remove this invariant, the program still executes perfectly well, but the proof using `loom_solve` no longer succeeds. The verifier simply does not have enough information to relate the current value of `cost` to the number of iterations completed so far.

At first glance, this may seem somewhat inconvenient because it requires more effort from the programmer. However, this is not necessarily a bad thing. Writing invariants is already a standard part of deductive verification, and in many cases the missing invariant becomes reasonably clear from the remaining goal state shown by Lean.

In practice, the unsolved goals and hypotheses displayed in the info panel often provide useful hints regarding what information is missing. This becomes particularly useful for discovering suitable cost invariants.

As another example, consider the following program which computes the integer square root using repeated subtraction. The algorithm relies on the identity

$$1 + 3 + 5 + \cdots + (2k - 1) = k^2.$$

The implementation is as follows.

```
method isqrt (mut n : Nat) return (r : Nat)
  ensures r * r ≤ nOld
  ensures nOld < (r + 1) * (r + 1)
  ensures nOld = r * r + n
  ensures cost ≥ costOld + 3 * r
  do
    let mut r := 0
    let mut odd := 1
    -- odd = 2*r + 1 will be maintained

    while odd ≤ n
    invariant nOld = r * r + n
    invariant odd = 2 * r + 1
    invariant r * r ≤ nOld
    do
      n := n - odd
      r := r + 1
      odd := odd + 2

    return r
```

Here the functional specifications express the usual correctness conditions for integer square root. The additional postcondition

```
ensures cost ≥ costOld + 3 * r
```

attempts to capture that each loop iteration performs roughly three meaningful updates.

Suppose now that we attempt verification using

```
prove_correct isqrt by
  loom_solve
```

The automation is unable to completely finish the proof and the goal panel shows the following remaining obligation.

```
unsolved goals
case «cost ≥ costOld + 3 * r»
nOld costOld cost n odd r : ℕ
invariant_1 : nOld = r * r + n
invariant_2 : odd = 2 * r + 1
invariant_3 : r * r ≤ nOld
done_1 : ¬odd ≤ n
i i_1 i_2 r_1 : ℕ
i_3 :
  { fst := cost, snd := { fst := n, snd := { fst := odd, snd := r } } } =
    { fst := i, snd := { fst := i_1, snd := { fst := i_2, snd := r_1 } } }
⊢ i ≥ costOld + 3 * r_1
```

Looking at this goal immediately suggests what is missing. The verifier is trying to prove a lower bound relating the current cost and the current value of `r`, but nowhere in the loop invariants have we told it how these two quantities are related inductively.

This naturally suggests adding the missing invariant

```
invariant cost ≥ costOld + 3 * r
```

Once this invariant is supplied, the verifier now has the inductive information required to carry the cost reasoning across loop iterations.

This illustrates a fairly common workflow in deductive verification. One often starts with a natural specification, invokes the verifier, and then examines the remaining goals to discover which invariants or hypotheses are missing. Cost-aware verification follows the same philosophy. The only difference is that we now also reason about the evolution of computational resources in addition to ordinary functional correctness.

# Chapter 6

# Limitations and future work

Like any verification system, the current implementation comes with several limitations. Most of these limitations are not fundamental problems with Loom or Velvet themselves but rather consequences of the particular design choices made in this thesis. In fact, many of them naturally suggest possible future directions and improvements.

## 6.1 Limitations

### 6.1.1 The cost model is syntactic

One important limitation is that the current cost model is fundamentally syntactic in nature. The implementation works by incrementing the ghost variable `cost` at selected syntactic constructs during elaboration. Therefore, the verified cost bounds correspond to this syntactic notion of cost rather than actual machine runtime.

For example, assignments, conditionals, recursive calls and loop iterations may increase the cost by some fixed amount according to how the elaborator was implemented. This gives a clean and mathematically manageable model, but it does not necessarily reflect the true runtime behavior on actual hardware.

Indeed, actual runtime depends on many external factors such as:

- compiler optimizations,
- memory layout,
- caching,
- processor architecture,
- garbage collection,
- operating system scheduling,

- and implementation details of data structures.

Our current model abstracts away all of these considerations. In this sense, the verified cost bounds should be viewed as reasoning about an abstract cost semantics rather than wall-clock execution time.

However, this is not entirely a disadvantage. Many formal systems reason using abstract cost semantics because exact runtime modeling is often extremely difficult and platform dependent. Even in complexity theory, we usually count abstract operations rather than physical execution time. The current framework follows a similar philosophy.

Still, one possible future direction would be to refine the cost model so that it more closely approximates actual execution cost.

### 6.1.2 Granularity of the cost model

Another limitation is that the current cost granularity is rather coarse.

At the moment, different operations are often assigned the same unit cost. For example, an assignment and a more computationally expensive operation may both increment the cost by one. Similarly, different branches or different kinds of recursive calls may contribute equally to the cost even if their actual runtime behavior differs significantly.

This simplification was intentional because the main goal of the thesis was to demonstrate that cost-aware verification can be integrated into Velvet with minimal changes to the existing infrastructure. A simpler cost model also keeps the elaboration process manageable and makes proofs easier to understand.

Nevertheless, this also means that the resulting cost bounds are only rough approximations. They provide useful structural information about program behavior but should not be interpreted as highly precise performance estimates.

A natural future extension would be to allow configurable cost semantics. For instance, different operations could contribute different weights:

- arithmetic operations,
- array accesses,
- recursive calls,
- allocations,
- branching,
- or even user-defined operations.

One could also imagine supporting multiple cost metrics simultaneously, such as:

- runtime cost,

- memory usage,
- energy consumption,
- blockchain gas usage,
- or communication cost in distributed systems.

The current implementation already suggests a path toward such extensions since the instrumentation happens systematically during elaboration.

### 6.1.3 Verification burden and loop invariants

Another practical limitation is the additional verification burden introduced by cost reasoning.

When verifying ordinary functional correctness, suitable loop invariants are already necessary. With cost-aware verification, programmers now also need to provide invariants describing how the cost evolves across iterations.

For example, one often writes invariants of the form:

```
invariant cost ≥ costOld + i
```

or more generally

```
invariant cost ≥ costOld + f(i)
```

where `f` describes how cost accumulates relative to the loop state.

This is not conceptually difficult, but it does increase annotation overhead. In more complicated programs, discovering suitable invariants can require some manual effort and familiarity with the verifier.

In many situations, however, the missing invariant becomes reasonably clear from the remaining goals shown by Lean after automation fails. The workflow then becomes iterative: attempt automation, inspect remaining goals, and strengthen the invariants accordingly.

Still, automatic inference of cost invariants would be a valuable future improvement. Techniques from abstract interpretation, SMT-based invariant synthesis or ranking-function generation may potentially be adapted for this purpose.

## 6.2 Future Work

There are several possible future directions following this work.

One immediate improvement would be better syntax support for recursive functions. As discussed earlier, recursive calls currently require manually threading the

`cost` variable through the program. While functionally correct, this becomes somewhat syntactically cumbersome. Since this is largely a surface-level issue involving elaboration and syntax sugar, it should be possible to hide much of this complexity from the programmer.

Another promising direction would be richer and more realistic cost semantics. Instead of uniform increments, the elaborator could assign different weights to different constructs or even support user-configurable cost policies.

It may also be possible to extend the framework toward asymptotic complexity verification. Rather than verifying only concrete lower or upper bounds, one could reason about asymptotic growth such as linear, logarithmic or polynomial behavior.

One particularly interesting direction is the connection to blockchain systems such as Ethereum, where computation is explicitly associated with gas costs. Since those systems already rely on resource accounting to guarantee bounded computation, a formally verified cost-aware framework may potentially be useful in reasoning about smart contracts and resource-bounded execution.

Finally, more extensive benchmarking and comparison against other verification systems would also be valuable. Since Loom and Velvet are relatively new systems, there remains substantial room for exploring their strengths, limitations and practical usability on larger verification case studies.

# Chapter 7

# Conclusion

In this thesis, we extended Velvet with a simple form of cost-aware verification. The main objective was not to develop a highly sophisticated cost analysis framework, but rather to demonstrate how a foundational verifier built inside Lean can be extended with relatively small modifications while still reusing the existing verification infrastructure.

The implementation works by introducing a ghost variable `cost` and instrumenting programs during elaboration so that cost is incremented at chosen program constructs. Because mutable variables in Velvet are automatically returned and can be referred to in postconditions and loop invariants, this provided a natural mechanism for reasoning about resource bounds without requiring major changes to the underlying verifier architecture.

One important aspect of the work is that the user-facing programming model remains largely unchanged. Programs are still written in the same imperative style provided by Velvet. Verification continues to use the same workflow through `prove_correct`, `loom_solve` and ordinary Lean tactics. In this sense, the extension integrates smoothly with the existing design of Loom and Velvet.

More broadly, the work highlights the flexibility provided by foundational verification frameworks and theorem provers such as Lean. Because Loom and Velvet are implemented directly inside Lean, extending the verifier becomes far more approachable than modifying large external verification pipelines. Much of the work can be done at the elaboration and metaprogramming layer while still retaining the guarantees provided by the underlying proof assistant.

In particular, the thesis demonstrates how syntax-directed transformations can be used to enrich a verification system with additional semantics. By modifying the elaboration phase, we were able to inject cost instrumentation automatically, allowing programs to support execution, testing and deductive verification of cost properties simultaneously.

Although the current implementation is intentionally simple, it points toward several interesting future directions such as richer cost semantics, automated invariant generation and verification of other resource properties such as memory usage or gas consumption.

Ultimately, perhaps the most important takeaway is not the particular cost model itself, but rather what the implementation demonstrates about Lean and foundational verification frameworks. The work shows that it is possible to extend a verifier with new reasoning capabilities while preserving much of the existing infrastructure, automation and proof workflow. In this sense, the thesis serves as a small case study in the extensibility and malleability of theorem-prover-based verification systems.

# Bibliography


[AdMKU25] Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. Theorem proving in lean 4. https://lean-lang.org/theorem_proving_in_lean4, 2025. Accessed: 2026-05-20.

[Chr24] David Thrane Christiansen. Functional programming in lean. https://lean-lang.org/functional_programming_in_lean, 2024. Accessed: 2026-05-20.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer, 2021.

[GPZ+26] Vladimir Gladshtein, George Pîrlea, Qiyuan Zhao, Vitaly Kurin, and Ilya Sergey. Foundational multi-modal program verifiers. *Proc. ACM Program. Lang.*, 10(POPL):2233–2264, 2026.

[Lea25a] Lean Forward. The hitchhiker's guide to logical verification. https://github.com/lean-forward/logical_verification_2025, 2025. Course materials and lecture notes on logical and formal verification, accessed May 21, 2026.

[Lea25b] Lean FRO. Lean language reference. https://lean-lang.org/doc/reference/latest, 2025. Accessed: 2026-05-20.

[PTA+25] Arthur Paulino, Damiano Testa, Edward Ayers, Evgenia Karunus, Henrik Böving, Jannis Limperg, Siddhartha Gadgil, and Siddharth Bhat. Metaprogramming in lean 4. https://leanprover-community.github.io/lean4-metaprogramming-book, 2025. Lean Prover Community. Accessed: 2026-05-20.

[Ver26] Verified Systems Engineering Lab. Velvet documentation. https://github.com/verse-lab/velvet/blob/master/velvet_

documentation.md, 2026. GitHub repository documentation, accessed May 20, 2026.